

Mixtec Parser Project: Developer Onboarding & Recent Changes Report

Table of Contents

1. Project Overview
 2. Directory and File Structure
 3. Core Components and Their Roles
 - `tokens.py`
 - `tree_node.py`
 - `parser.py`
 - `interpreter.py`
 - `test_xml_interpretation.py`
 4. How Parsing and Interpretation Work
 5. Recent Changes: XML Interpretation & Testing
 6. How to Run and Extend the Tests
 7. Summary of Key Methods and Classes
 8. Next Steps and Recommendations
-

1. Project Overview

This project is a parser and interpreter for a limited grammar inspired by Mixtec codices. It uses a custom tokenization and parsing system, with support for ANTLR-based parsing. The main goal is to convert a sequence of tokens (representing elements like humans, years, objects, etc.) into both a human-readable interpretation and an XML representation.

2. Directory and File Structure

- `tokens.py` — Defines the basic token types (e.g., Human, Year, Obj).
 - `tree_node.py` — Defines the parse tree node classes and logic for interpretation.
 - `parser.py` — Implements the recursive descent parser that builds the parse tree.
 - `interpreter.py` — Provides a simple interface for interpreting the parse tree.
 - `test_xml_interpretation.py` — Contains tests for XML output and parser behavior.
 - `Scene*.py`, `Scene.g4`, etc. — ANTLR-generated files and grammar (not the focus of recent changes).
 - `README.md` — Project documentation.
-

3. Core Components and Their Roles

tokens.py

Path: tokens.py

Defines the basic building blocks (tokens) for the parser. Each token represents a concept from the Mixtec codices, such as a human figure, a year, or an object.

Key Classes: - **Token:** Base class for all tokens. Now only stores **x** and **y** coordinates. - **Human, Year, NameDate, Obj, NearObj, End:** Subclasses for specific token types, each with relevant attributes. - **New:** Each token class now has a **to_xml()** method that returns an XML string representing the token.

Example:

```
class Human(Token):
    def __init__(self, x, y, gender, pose, orientation):
        ...
    def to_xml(self) -> str:
        return f'<human x="{self.x}" y="{self.y}" ... />'
```

tree_node.py

Path: tree_node.py

Defines the structure of the parse tree and how to interpret it.

Key Classes: - **TreeNode:** Abstract base class for all tree nodes. - **Start, Sent, Clause, DateTail, ObjTail, Date, ClauseF, NearDate:** Internal nodes representing grammar rules. - **New:** **LeafNode:** Wraps a **Token** so it can be used as a tree leaf. Implements **interpret()** and **to_xml()** by delegating to the token.

Why LeafNode? - Ensures all children in the parse tree are **TreeNode** instances, fixing bugs from earlier dynamic inheritance.

parser.py

Path: parser.py

Implements a recursive descent parser that takes a list of tokens and builds a parse tree using the classes from **tree_node.py**.

Key Points: - Uses helper methods like **consume_next_token()** to add tokens as **LeafNode** instances. - Each grammar rule (e.g., **sent**, **clause**) is a method that builds part of the tree.

Example:

```
def consume_next_token(self, c, exp):
    token = self.tks[-1]
    c.append(LeafNode(token))
    self.consume(exp)
```

interpreter.py

Path: interpreter.py

Provides a simple interface for interpreting the parse tree.

Key Class: - **Interpreter:** Calls the `interpret()` method on the root of the parse tree.

test_xml_interpretation.py

Path: test_xml_interpretation.py

A comprehensive test suite for the new XML functionality and parser behavior.

Key Functions: - `test_individual_token_xml()`: Checks that each token's `to_xml()` method works. - `test_xml_generation_from_samples()`: Generates XML for sample token lists. - `test_parser_with_xml_output()`: Runs the parser and interpreter, showing both XML and human-readable output.

How to run:

```
python test_xml_interpretation.py
```

4. How Parsing and Interpretation Work

1. **Tokenization:** Tokens are created (e.g., using `get_sample_tokens()`).
 2. **Parsing:** The **Parser** takes a list of tokens and builds a tree of **TreeNode** and **LeafNode** objects.
 3. **Interpretation:** The **Interpreter** walks the tree, calling `interpret()` on each node to produce a human-readable string.
 4. **XML Output:** Each token can be converted to XML using its `to_xml()` method. The test file demonstrates how to generate a full XML scene.
-

5. Recent Changes: XML Interpretation & Testing

- Added `to_xml()` methods to all token classes in `tokens.py`.
- Introduced **LeafNode** in `tree_node.py` to wrap tokens for use in the parse tree.

- **Updated the parser** (`parser.py`) to always wrap tokens in `LeafNode`.
 - **Created `test_xml_interpretation.py`** to test XML output and parser behavior.
 - **Fixed bugs** related to dynamic inheritance and tree structure.
-

6. How to Run and Extend the Tests

- Run all tests:

```
python test_xml_interpretation.py
```
 - To add new tokens or test cases, update the `get_sample_tokens()` and `construct_from_samples()` functions in the test file.
 - To add new XML output logic, extend the `to_xml()` methods in the relevant token or node classes.
-

7. Summary of Key Methods and Classes

- **`tokens.Token` and subclasses:** Represent the basic elements of the scene.
 - **`tokens.Token.to_xml()`:** Returns an XML string for the token.
 - **`tree_node.LeafNode`:** Wraps a token for use in the parse tree.
 - **`parser.Parser`:** Builds the parse tree from a list of tokens.
 - **`interpreter.Interpreter`:** Produces a human-readable interpretation from the parse tree.
 - **`test_xml_interpretation.py`:** Tests all of the above.
-

8. Next Steps and Recommendations

- Consider adding XML output for entire scenes (not just individual tokens).
 - Integrate XML output into the ANTLR-based visitor if needed.
 - Expand the test suite with more complex scenes and edge cases.
 - Update the `README.md` to reflect these changes and provide usage instructions.
-

End of Report